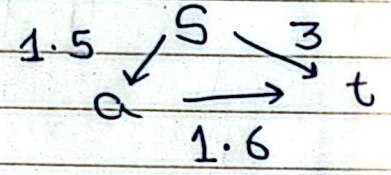
Chapter 4 - Exercises

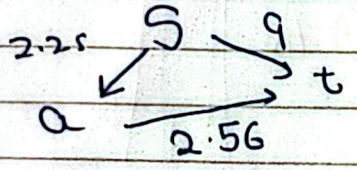
1) The statement is true. Let e^* be (x, y) and let S be the set of all edges nodes except X. Then $V - S = \{x\}$ and so e^* is the minimum cost edge with one end in S and the other in $V-S$, and so by the cut property e^* is in every MST.

2)a) The statement is true. Since all the edge costs are distinct, there is only one MST, and Kruskals algorith will add edges in the same order because for any pair of edge costs ce_1 and ce_2, if $ce_1 < ce_2$ then $ce_1^2 < ce_2^2$ as before squaring

b) The statement is ~~true~~ false. Suppose we have the graph:-



The shortest path is $s \to t$. However if we square each edge cost we get:-



Now the shortest path is $s \to a \to t$.

using our algorithm

3) We must prove that ~~using~~ the accumulated total number of packages we have shipped by the time the i^{th} truck completes its journey (i.e. P_i) is ~~greater~~ atleast as much as the same metric under any other algorithm (i.e. Q_i). In other words, $P_i \geq Q_i$ $\forall\ i \geq 1$.

~~If~~ $Q_j \leq P_j$ for $j=1$ because ~~with~~ we fill each truck with as many boxes as possible.

Now suppose ~~Q_k~~ $j > 1$ and $Q_i \leq P_i$ $\forall i < j$. Then we have:-

$$Q_{j-1} \leq P_{j-1}.$$

If $Q_{j-1} = P_{j-1}$, then $Q_j \leq P_j$ because they ~~next~~ have the same boxes to load next, (since boxes must be loaded in order) ~~and they have loa~~ and P_j loads as many of the boxes next in line as possible. ~~If $Q_{j-1} \leq P_{j-1}$, then while loading the j^{th} truck, either its loa~~ ~~So~~ If $Q_{j-1} < P_{j-1}$, then to get $Q_j > P_j$, the j^{th} truck in the alternate algorithm will have to load everything the j^{th} truck in our algorith does, and more additional boxes that come after P_j too, but that contradicts the fact that ~~we~~ we load as many boxes in our algorithm as we can, ~~ie~~ and so $Q_j \leq P_j$.

It follows that with our algorithm, ~~is optimal~~ we exhaust all boxes with at most as many trucks as any other algorithm, hence our solution is optimal.

5) We sort all house positions from west to east. Let i be the first uncovered house at position x, ~~th~~ miles from the start, then we put a base station at x+4 miles, thus covering all houses in the range [x, x+8]. ~~If the~~ Skip ahead till x+8, then ~~or~~ repeat, placing a base station 4 miles ahead of the next ~~house~~ uncovered house and so on, until every house is covered. If for any ~~the~~ house, x+4 is past the endpoint, put the base at the endpoint instead.

Let $S = \{s_1, s_2, \dots s_n\}$ denote the set of base stations positions we placed with our algorithm, and $T = \{t_1, t_2 \dots t_m\}$ the set of base stations place by any other algorith, sorted in ~~an east to~~ the same order as our algorith ran (e.g. east to west if we started at the eastern end).

We know $s_i \geq t_i$ because we moved as far as possible from the startpoint before placing the first base station (while ensuring all previous houses are covered). Suppose for some $i \geq 1$, $s_i \geq t_i$, i.e. $s_1 \cdots s_i$ cover all the house $t_1 \cdots t_i$ covers. We know $s_1 \cdots s_i, t_{i+1}$ will hence not leave any uncovered houses to the ~~left of~~ b/w s_i and t_{i+1}, but our algorith maximise s_{i+1}, so $s_{i+1} \geq t_{i+1}$.

Now suppose $k > m$, then $\{s_1 \cdots s_m\}$ does not cover all houses, but $s_m \geq t_m$ and so $\{t_1 \cdots t_m\}$ also fails to cover all houses i.e. a contradiction therefore $k \leq m$

6) Sort the runners in descending ordering in terms of T_i, the sum of their projected biking time and projected running time, and send them out in that order.

To prove this greedy choice minimises completion time, let O be any other ordering such that O is optimal. Then O contains an inversion and thus there exists x, y in O such that x and y are next to each other, but $T_x \leq T_y$ while x comes

before y, where T_i is the sum of projected running and biking time for comper i.

Let P be the point in time when x gets out of the pool. The the completion times of x and y are:-

$$C_x = P + S_x + T_x$$
$$C_y = P + S_x + S_y + T_y.$$

Since $T_y > T_x$, $C_y > C_x$. If we exchange the positions of x and y in the sorting, we get:-

$$C'_x = P + S_y + S_x + T_x$$
$$C'_y = P + S_y + T_y.$$

with x and y next to each other

Now, $C'_x < C_y$ because $T_x < T_y$, and $C'_y < C_y$, therefore $\max(C'_x, C'_y) < \max(C_x, C_y)$ so removing the inversion does not increase completion time. Repeating this for every inversion, hence gives us the solution returned by our greedy algorithm without sacrificing optimality, hence our algorithm is optimal.

The running time of this algorithm is $O(n \log n)$ since it only involves a single simple sort.

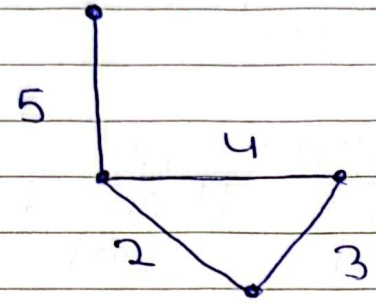
8) We ~~run the jobs~~ Let G be any connected graph with distinct edge costs.

Suppose G does not have a unique MST. Then $\exists$ two MSTs T_1 and T_2 such that $T_1 \neq T_2$.

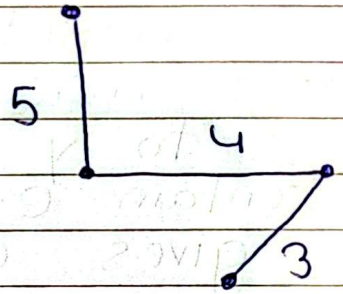
Since $T_1 \neq T_2$, $\exists$ an edge $e = (x, y)$ in T_1 that does not exist in T_2. Since T_2 is by definition connected, that means there is a path from x to y in T_2 that does not contain e. Adding e to that path gives a cycle that exists in G.

By the cycle property, and the fact that all edge costs are unique, $\exists$ a unique highest cost edge in this cycle that does not exist in any MST. But edge e is in T_1 and the remaing edges of the cycle is in T_2, so one of T_1 or T_2 is not an MST, which contradicts our supposition. Therefore G has a unique MST.

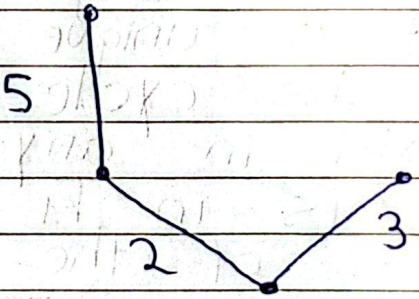
9)a) Let G be the graph:-



Then an MBT would be:-



But that MBT is not an MST because the only MST for this graph is:-



b)

b) Let $G = (V, E)$ be any connected graph with distinct edge costs, and suppose T_1 is an MST of G.

for sake of contradiction,
Suppose T_1 is not an MBT of G and let $e = (x, y)$ be its bottleneck edge.

Then $\exists$ a spanning tree T_2 of G such that the bottleneck edge of T_2 has cost less than e, and so $e \notin T_2$. But T_2 is a spanning tree, so $\exists$ a path in T_2 from x to y. G contains both this path and edge e, thus forming a cycle in G.

The path only has edges with cost less than e (otherwise T_2 would not have a bottleneck edge cost lower than e) and so e is the highest cost edge in a cycle of G. But by the cycle property, e hence does not belong to any MST of G, yet $e \in T_1$, thus implying T_1 is not an MST, i.e. contradicting our supposition. Therefore T_1 is an MBT of G.

10)a) Suppose $e=(x,y)$ is the new edge added to G. ~~If~~ We then check if the cost of e is greater than every edge in the path from x to y in T. If it is, then by the cycle property, e is not in any MST, else if it is not then T is still an MST. This check requires a graph traversal algorithm on T, which is implemented as an adjacency list. The ~~time is~~ running time is hence $O(|E|+|V|)$, however, since T is a tree, $|E|=|V|-1$ so the running time is $O(|V|)$.

b) We ~~simply~~ add the edge $e=(x,y)$ to T. Now we use a graph traversal algorithm to find and delete the highest cost edge ~~in C~~ (or any maximum edge if there are multiple) in $C-\{e\}$, where C is the ~~set~~ set of edges forming a cycle when we add e to T.

~~This Adding e to T takes time O(|V|) if we are using T Assuming an Arraylist Assuming we use an~~
The running time is $O(|E|)$ because we use a simple graph traversal algorith on $T+\{e\}$, which has $|V|=|E|$ so $O(|V|+|E|)$ becomes $O(|E|)$.

13) We sort the jobs in descending order in terms of w_i / t_i.

Let O be an optimal solution that is different from the solution returned by our greedy algorithm. Then $\exists$ a pair of jobs j_x and j_y such that j_x is directly followed by j_y but $w_x / t_x \leq w_y / t_y$. We call such a pair an 'inversion'.

Let C be the completion time of the job directly before j_x. Then we have:-

$$C_x = C + t_x$$
$$C_y = C + t_x + t_y$$

And their weighted sums are $w_x C_x$ and $w_y C_y \Rightarrow w_x C_x + w_y C_y$

Suppose we remove the inversion by exchanging the positions of j_x and j_y. Then we have:-

$$C'_x = C + t_y + t_x$$
$$C'_y = C + t_y$$

And their weighted sums are $w_x C'_x$ and $w_y C'_y \Rightarrow w_x C'_x + w_y C'_y$.

To show the weighted sum does not increase by removing an inversion, we show the following expression is non negative:-

$$(w_x C'_x + w_y C'_y) - (w_y C'_y + w_x C'_x)$$

$$= w_x C_x + w_y C_y - w_x C'_x - w_y C'_y$$

$$= w_x(C + t_x) + w_y(C + t_x + t_y) - w_x(C + t_y + t_x) - w_y(C + t_y)$$

$$= w_x C + w_x t_x + w_y C + w_y t_y + w_y t_x - w_x C - w_x t_x - w_x t_y - w_y C - w_y t_y$$

$$= w_y t_x - w_x t_y$$

We know $w_x / t_x \leq w_y / t_y$ so $w_x t_y \leq w_y t_x$, therefore $w_y t_x - w_x t_y \geq 0$ ∎

∴ we have shown that removing an inversion does not increases the weighted sum of the completion times; while the weighted sum of the jobs before and after stay the same, so the ~~weighted~~ by removing all inversions, we can transform O into the solution returned by our algorithm without saerificing optimality hence proving our solution is optimal.

14(a) We sort the finish times in ascending order. ~~Then we~~ Initially all the processes are unchecked, we run status check at the earliest finish time among unchecked processes and then mark all process with said finish time in their 'start to finish' interval as checked. We stop once there are no unchecked processes left.

Let C be the set of times at which our algorithm ran status_check and C' be ~~the same for~~ any other valid set of intervals. The ~~first~~ first status_check in C is at the latest possible time while ensuring all processes that started before it gets checked, so by that time C' has ~~ran~~ ran status_check at least once as well. Suppose at ~~time~~ the time when C runs status_check for the k^{th} time, C' has run it at least k times as well. The process at which's finish time C will run ~~k~~ status_check for the ~~k~~ $(k+1)^{th}$ time had not started when C ran status_check for ~~the~~ k^{th} time, so C' has not checked it either at that point, and so C' will have to also run status_check ~~between the~~ k^{th} time at or before ~~the~~ ~~(k+1)st~~ time when C runs its $(k+1)^{st}$ status_check, so

even at time when C runs its $(k+1)^{st}$ status_check, C' will have run at least k+1 status_checks as well. Hence C is an optimal solution as it minimises the number of status_checks

Sorting the process in terms of finish times takes time $O(n \log n)$, and then a pass taking time $O(n)$ is needed to select all the times to run status_check at, thus giving overall $O(n \log n)$ time.

b) Suppose k^* is the maximum number such that we can find k^* mutually disjoint processes.

Let C be the set of times at which we ran status_check, returned by our algorithm from part a. $\forall c_i \in C$, $c_i = f(P_x)$ (i.e every time in C is the finish time of some process P_x). Also, every process that caused status_check to be run is mutually disjoint from any other set that did the same, else if they overlapped then our algorithm would have checked both with a single status_check. Hence $|C| =$ the size of some set of mutually disjoint jobs, and since

k^* is the size of the largest of all such sets, $|C| \leq k^*$.
Hence we can choose the ~~set of~~ times in C and then add more random times until we have k^* times to get a set of k^* times ~~that~~ at which we can run status_check to and cover all sensitive processes.
□

18) Dijkstra's Algorithm Edited (G, l)
Let S be the set of explored nodes
For each node $u \in S$ we store a ~~set distance~~ time $d(u)$ and a pointer → $p(u)$ to the node from which we last 'relaxed' u.
Initially $S = \{s\}$, $d(s) = 0$ and $p(s) = \text{null}$
While $S \neq V$
Select a node $v \notin S$ ~~such that~~ with atleast one edge from S such $d'(v) = \min_{e=(u,v): u \in S} f_e(d(u))$ is minimum.
Add v to S and let $d(v) = d'(v)$ and $p(v) = u$
Endwhile.

Since this is simply Dijkstra's algorithm except we use a constant time cost function for to get the time from the start time to a node u on the current path (rather than simply doing $d(u) = t + l_e$ where l_e is the

length of edge e), the running time is same i.e. O(m log n). Recreating the path from u to starting point requires traversing through at most m pointers i.e. O(m) so running time stays O(m log n).

19) Let G = (V, E) be any connected graph, and $\forall e \in E$, define the cost $c(e) = 1/b_e$.

Then for any nodes u and v, the path with the best achievable bottleneck rate is the same as the path for which the maximum cost edge is minimal.

Let T be any MST of G using the above definition of edge costs. Then T has a unique path between any two nodes u and v, let that path be P. Also suppose there exists a path P' in G such that $P' \neq P$ and P' has a better BABR (i.e. its maximum cost edge has a lower cost than P's maximum cost edge).

Delete any of P's maximum cost edges 'e' from T. Then

T is broken into two connected components, and by the cut property, there is no edge in G connecting both components with cost less than e. Now add P' to T-{e}, thus we reconnect both components. But the edge e' in P' that reconnects them has a lower cost than e, i.e. a contradiction. Hence every unique v-u path in T has BABR at least as good as any other u-v path in G.

Setting all edge costs to 1/be would take time O(E) and then Kruskal / Prims to find an MST would take time O(mlogn) for an overall running time of O(mlogn).

a)

20) The first conjecture is true. It is the same problem as in exercise 19, hence refer there for the proof, with the only difference being that we have the edge costs are the same as the altitude values rather than its reciprocal that here the edge cost = the altitude values rather than taking its reciprocal as we did in question 19.

b) Only if direction

Suppose $G' = (V, E')$ is a minimum altitude subgraph of G and T is the MST of G.

Suppose there exists an edge $e = (x, y)$ in T such that $e \notin E'$. Then there must be path(s) in G' from x to y. Let P be the winter optimal path among them.

If we delete e from T, we get two disconnected components. By the cut property, there is no edge with lower altitude than e that connects these components. If we now add P to $T - \{e\}$, it will be connected again, and there must be an edge e' in P that connects them.

$a_{e'} < a_e$ is not possible, because e is what is implied by and neither is $a_{e'} = a_e$ because all altitudes are distinct. Hence we have $a_{e'} > a_e$. Now a_e is a path in G from x to y

with lower height than P, but this contradicts the fact that G' is a minimum altitude connected subgraph. Hence our supposition is false i.e there is no edge e in T such that $e^* \notin E'$.

If direction

$\rightarrow = (V, E')$

Let G' be any connected subgraph such that it contains all the edges of the MST

By the first conjecture, which is already proven true, T is itself a minimum altitude subgraph of G. Also, Let x and y be any two nodes. Since T is a subgraph of G', the (x, y) path in T has height at most any x, y path in G'. And Since G' is also a subgraph of G, T is also a subgraph

Let $L_H(x, y)$ be the height of the minimum height path from x to y in subgraph H. Since T is a subgraph of G', every path in T is available in G', so $L_{G'}(x, y) \leq L_T(x, y)$ because the best path in G' cannot be worse than in T.

C_{10}

Also, since T is a minimum altitude connected subgraph of G, $L_T(x,y) \leq L_G(x,y)$.
Hence we have:-

$$L_{G'}(x,y) \leq L_T(x,y) \leq L_G(x,y)$$

Therefore $L_{G'}(x,y) \leq L_G(x,y)$ for any pair of nodes, so G' is a connected subgraph.

11) Suppose $G=(V,E)$ is a connected weighted graph and T is an MST of it.

Let X be any valid ordering of the edges of G. Now at every sequences of edges in X where the edges have the same cost, change it so that the edges in T are at the front of that sequence. We still have a valid ordering because the edge costs remain non decreasing.

When we run Kruskal's algorithm with X, we get T because every edge of T will be accepted because adding an edge of T to previously chosen

edges of T cannot create a cycle.

Consider any edge $e = (u, v) \notin T$. In T there is a unique path from u to v with cost at most $c(e)$ because otherwise replacing an edge with cost more than e would give with e would give a cheaper spanning tree thus contradicting the fact that T is an MST.

All edges on the u-v path are processed before e because smaller cost edges come earlier, and equal cost edges here in T were placed before the ones not in T. Since those edges are already accepted, u and v are already connected when e is considered so e is rejected.

∴ this execution outputs T.